

BitSLAM: Trajectory-Based SLAM Using Particle Filters

Gabriel J. Ferrer¹ and Jacob Collier-Tenison²

Abstract—Solutions to the trajectory SLAM problem enable inexpensive robots with minimal sensors to build accurate maps of their environments. We present BitSLAM, which addresses the trajectory SLAM problem by employing a particle filter and a compact map representation. We assess in both simulated and physical indoor environments the extent to which BitSLAM is a viable solution to the trajectory SLAM problem.

I. ALGORITHM

Chen *et al* of iRobot investigated SLAM with pose graphs derived solely from the robot’s trajectory, without employing range-based sensors or computer vision to obtain landmarks [3]. A hallmark of their approach, as is typical in modern SLAM-based systems, is the need for loop-closure. Particle filters have long been employed in solving the SLAM problem [1] [2] employing laser rangefinders. Using particle filters for trajectory-based SLAM would eliminate the requirement for explicit loop closure by retaining many particles as hypotheses of the system’s state, meaning that each observation is compared to many particles rather than reconciled with a single belief-state.

We introduce BitSLAM, a SLAM algorithm based on particle filters. Each particle has an estimate of the robot’s pose and a map of the robot’s environment. The pose is an (x, y, θ) triple, where x and y represent the robot’s distance in meters from its starting point and θ represents the robot’s angular distance in radians from its initial heading. All values are directly calculated from the robot’s odometry sensors.

BitSLAM maps contain two bit vectors. The obstacle map contains a 1 for every identified obstacle while the space map contains a 1 for every location the robot visits. Locations are 10 cm^2 . Bits are packed into 64-bit machine words. Inspired by Chess bitboards [4], this enables compact storage of thousands of particles. Maps resize as needed.

Each particle is updated every time odometry is received and whenever sensors detect a collision. We first add noise to each particle. If the robot was moving through open space, we update the particle’s space map. If the robot struck an obstacle we update the particle’s obstacle map. We set separate noise levels for when the robot hits an obstacle as opposed to when it is moving in free space.

When an obstacle in the obstacle map is enveloped by spaces in the space map, the map is no longer consistent with the physical environment. In a *consistent map*, every obstacle has one, two, or three Manhattan neighbors with a value of 1 in the space map. Particles with inconsistent maps

are eliminated. The filter repopulates by resampling from the consistent particles. Resampling weights are calculated for each consistent particle in two ways:

- **MinPose**: Find the minimum distance between the particle’s estimated pose and the estimated poses of all eliminated inconsistent particles.
- **OdometryGap**: Find the distance between the particle’s estimated pose and the raw odometry estimate.

II. PHYSICAL EXPERIMENT SETUP

We are developing BitSLAM using ROS2 Iron Irwini aboard an iRobot Create3 robot connected via USB-C to a Raspberry Pi. BitSLAM is written in Rust.

Path A results from running our robot with a simple obstacle-avoiding algorithm in an 3x4 meter empty room. At the end of the 4-minute run we measured the robot’s physical displacement from its starting point (1.8 meters).

Path B results from using BitSLAM to map a second room containing a couple of tables, one along a wall and another in the center. It drove straight until encountering an obstacle, turning until on a path to an unvisited map location. Displacement from start after a 4:40 run was 1.28 meters.

It is not possible to create a consistent map of either **Path A** or **Path B** from raw odometry. For **Path A**, with 4878 steps, an inconsistent map is reached after 1572 steps. **Path B**, with 5898 steps, fails after 177 steps.

We created the noise model from 500 data points; 100 data points from each of five different environments with different surfaces. To optimize the collision noise parameters, we tested the particle filter at linear multiples of the average collision offsets derived from the aforementioned data. For collisions, we settled on using 0.032 m for x/y noise and 0.62 radians for θ noise. For odometry updates, we used 7×10^{-4} m for x/y noise and 2×10^{-4} radians for θ noise.

III. PHYSICAL EXPERIMENT RESULTS

For each of eight combinations of particle count (1000 vs 2000), weight strategy (**MinPose** vs **OdometryGap**), and physical-environment odometry (**Path A** vs **Path B**) we ran 100 map-generation experiments (data is in Table I). Values in brackets represent bounds for a 95% confidence interval, using Wilson’s method for number of failures.

Except for **Path B** using **OdometryGap**, all experiments always had consistent maps. Increasing the number of particles greatly reduced the number of failures in that case, but otherwise 2000 particles presented no significant advantage.

In all cases the maximum time needed for one iteration of the particle filter was less than 10 milliseconds, plenty of

¹Gabriel Ferrer, Professor of Computer Science, Hendrix College, Conway, AR 72032, USA ferrer@hendrix.edu

²Jacob Collier-Tenison, Computer Science major, Hendrix College, Conway, AR 72032, USA colliertenisonjt@hendrix.edu

Path	Particles	Weighting	Failures	Max Iteration (ms)	Closest Particle Rank	Closest to Actual	Highest-Ranked to Actual
A	1000	MinPose	0	[2.03,2.22]	[600,778]	[0.64,0.75]	[6.22:6.77]
A	1000	OdometryGap	0	[1.91,2.35]	[993,994]	[0.79:0.89]	[1.80,1.80]
A	2000	MinPose	0	[4.03,4.92]	[975,1364]	[0.45,0.53]	[6.83,7.59]
A	2000	OdometryGap	0	[6.71,7.57]	[1989,1991]	[0.69,0.76]	[1.80,1.80]
B	1000	MinPose	0	[3.23,3.67]	[422,596]	[0.28,0.34]	[2.41,2.68]
B	1000	OdometryGap	[6,18]	[3.56,3.82]	[868,901]	[0.48,0.54]	[1.20,1.28]
B	2000	MinPose	0	[6.64,6.76]	[954,1256]	[0.24,0.29]	[2.60,2.85]
B	2000	OdometryGap	1	[6.57,6.89]	[1783,1811]	[0.49,0.54]	[1.23,1.28]

TABLE I
REAL-WORLD RESULTS

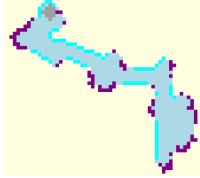


Fig. 1. Path B, MinPose



Fig. 2. Path B, OdometryGap

time for incorporation into a real-time robot controller. In fact, we did so when collecting **Path B**.

The the weight strategies differ drastically with regard to distance from the physically measured final location. Consistent with its design, the distance from the estimate to the measured location for **OdometryGap** is similar to the distance from the raw odometry to the measured location. The advantage of using BitSLAM, then, is a consistent map to accompany that estimate. Intriguingly, the closest estimate among the particles is much closer. Identifying the closest particle more effectively remains an open question.

MinPose performed poorly in comparison. Figure 1 shows that **MinPose** exhibits a bias towards maps that redundantly represent previously visited areas. In contrast, Figure 2 demonstrates that **OdometryGap** can map a looped path around a central obstacle.

IV. SLAM TOOLBOX COMPARISON

We further evaluated BitSLAM with the `hall_04` sequence from the M2DGR dataset [5]. As M2DGR does not

Odometry Source	RMSE (m)	Confidence Interval
Raw	3.38	[3.09, 3.27]
1000 particles	3.37	[3.08, 3.25]
2000 particles	3.48	[3.09, 3.33]
3000 particles	3.35	[3.03, 3.23]
SLAM Toolbox	1.53	[1.12, 1.48]

TABLE II
SLAM TOOLBOX COMPARISON

include an odometry estimate, we produced one from the laser scans using RF2O Laser Odometry [6]. We used the resulting laser scan and odometry topics as input to both BitSLAM and SLAM Toolbox [7]. Results are in Table II.

BitSLAM (using **OdometryGap**) did not improve upon the raw odometry estimate, falling well short of SLAM Toolbox. Increasing the number of particles made no significant impact. BitSLAM struggles to take significant steps away from the underlying odometry, undoubtedly due to **OdometryGap**. However, the good performance of **OdometryGap** in earlier experiments raises the question of designing a better heuristic that will allow for the map to diverge from the underlying odometry towards the unknown ground truth.

V. CONCLUSION

BitSLAM is a viable SLAM algorithm in small, mildly cluttered indoor spaces for robots using bump sensors. It maintains a population of 2000 particles, each with its own map, in under 10 ms per iteration. It sidesteps the loop closure problem by maintaining a large population of maps and pose hypotheses. It is a less viable solution to more general SLAM problems using distance sensors. Future work will explore the use of inexpensive distance sensors and refinements to the **OdometryGap** strategy.

REFERENCES

- [1] A. I. Eliezer and R. Parr, “Hierarchical Linear/Constant Time SLAM Using Particle Filters for Dense Maps,” 2005 Conference on Neural Information Processing Systems.
- [2] M. Montemerlo and S. Thrun, “Simultaneous localization and mapping with unknown data association using FastSLAM,” IEEE ICRA, Taipei, Taiwan, 2003, pp. 1985-1991.
- [3] Y. Chen et al., “Trajectory-Based SLAM for Indoor Mobile Robots with Limited Sensing Capabilities,” 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Detroit, MI, USA, 2023, pp. 9147-9154, doi: 10.1109/IROS55552.2023.10341518.
- [4] G. Adelson-Velsky, V. Arlazarov, A. Bitman, A. Zhivotovsky, A. Uskov (1970). “Programming a Computer to Play Chess”. Russian Mathematical Surveys, Vol. 25, pp. 221-262.
- [5] Jie Yin, “M2DGR: a Multi-modal and Multi-scenario SLAM Dataset for Ground Robots”, <https://github.com/SJTU-ViSYS/M2DGR>. Retrieved August 10, 2026.
- [6] Mariano Jaimez and Javier G. Monroy and Javier Gonzalez-Jimenez, “RF2O Laser Odometry”, https://github.com/MAPIRlab/rf2o_laser_odometry. Retrieved August 10, 2026.
- [7] S. Macenski and I. Jambreci, “SLAM Toolbox: SLAM for the dynamic world”, Journal of Open-Source Software 6(61), 2021, https://github.com/SteveMacenski/slam_toolbox. Retrieved August 10, 2026.